

Chapter2 Intro

Introduction to Web Application Development Technologies

一、Framework

- A **framework** is a big library or group of libraries that provides many services (rather than perhaps only one focused ability as most libraries do).
- Examples:
 - **Django** - a full-stack server-side web application framework built using **python**
 - **Ruby on Rails** - a full-stack server-side web application framework built using **ruby**
 - **Laravel** - a free, open-source **PHP** web framework, following the model-view-controller (MVC) architectural pattern.
 - **ASP.NET** - a server-side web application framework developed by **Microsoft**.
 - **Bootstrap** - a free and open-source CSS framework directed at responsive, mobile-first front-end web development with **HTML, CSS and Javascript**

A framework is a big **library** or group of libraries that **provides many services**.

框架是预先编写的代码库或模板，开发人员使用它们来更高效地构建Web应用程序。提供了现成的代码、库和功能，减少了重复劳动。

Examples:

- **Django** - a full-stack server-side web application framework built using **python**
- **Ruby on Rails** - a full-stack server-side web application framework built using **ruby**
- **Laravel** - a free, open-source **PHP** web framework, following the model-view-controller (MVC) architectural pattern.
- **ASP.NET** - a server-side web application framework developed by **Microsoft**.
- **Bootstrap** - a free and open-source CSS framework directed at responsive, mobile-first front-end web development with **HTML, CSS and Javascript**

▼ 解释

Django – a full-stack server-side web application framework built using **python**

一个基于Python构建的**全栈服务器端**Web应用程序框架。Django强调快速开发和干净、实用的设计。

1. 基于Python构建: Python开发。
2. 全栈: 从前端到后端所有开发任务。
 - a. 前端: 用户界面、用户体验
 - b. 后端: 服务器端逻辑、数据库管理...
3. 服务器端: 指在服务器上运行的代码和逻辑, 一般用于处理数据请求、存储数据以及执行业务逻辑。
4. 应用程序框架: 预先编写的代码库和工具, 提供了应用程序开发的基础结构

区别点	服务器端	客户端
运行位置	服务器	用户设备 (如浏览器、移动应用)
主要职责	处理请求、业务逻辑、数据存储	用户界面显示、交互处理、发送请求
典型技术	Node.js、Django、ASP.NET等	HTML、CSS、JavaScript等
示例	生成网页内容并发送给客户端	显示网页内容并处理用户交互

网页中的内容如何在浏览器中显示?

网页加载过程

1. **请求网页**: 当用户点击一个链接或输入一个URL时, 浏览器会向相应的Web服务器发送一个HTTP请求。这请求是由用户的设备 (客户端) 发出的。
2. **服务器响应**: Web服务器接收到请求后, 会处理该请求, 并返回相应的网页内容。这个内容通常是HTML文件, 以及可能包含的CSS、JavaScript、图片等资源。
3. **浏览器解析和渲染**: 浏览器接收到服务器返回的内容后, 会解析HTML文件, 并通过CSS进行样式设置, 使用JavaScript执行客户端逻辑, 最终在用户的设备上渲染出网页。

浏览器的作用

- **浏览器 (Client)**: 是用户访问和显示网页的工具。当你在浏览器中打开一个网页时, 浏览器会负责请求服务器上的资源、解析内容并在屏幕上呈现出来。
- **本地缓存**: 为了提高加载速度, 浏览器会将一些资源 (如图片、CSS文件) 存储在本地缓

存中。当用户再次访问同一网页时，这些资源可以直接从本地缓存中加载，而不是重新从服务器请求。

全栈与服务器端是否矛盾？No！

Django主后端开发，但是在服务器端可以实现部分前端的开发

- **模板引擎**：Django的模板引擎允许在服务器端生成动态HTML内容，这样的前端工作是由Django完成的。
- **管理界面**：Django自带的管理界面（Admin Interface）提供了一个用于数据管理的前端界面，这部分前端工作也是由Django提供的。

Ruby on Rails – a full-stack server-side web application framework built using [ruby](#)

一个基于Ruby构建的**全栈服务器端**Web应用程序框架。Rails以其简洁和高效的开发方式著称。

```
▼ Ruby |
1  # 这是一个简单的 Ruby 代码示例
2  def greet(name)
3    puts "Hello, #{name}!"
4  end
5  greet("World") # 输出: Hello, World!
```

- **Web开发**：使用 Ruby on Rails 构建动态的Web应用程序。
- **脚本编写**：使用 Ruby 编写自动化脚本和工具，提高工作效率。
- **数据处理**：Ruby 拥有强大的数据处理和分析库，可以用于处理数据。

Laravel – a **free, open-source PHP** web framework, following the model-view controller (MVC) architectural pattern.

一个免费的开源PHP Web框架，遵循模型-视图-控制器（MVC）架构模式。Laravel提供了优雅的语法和强大的工具集。

PHP（Hypertext Preprocessor）是一种广泛使用的开源脚本语言，特别适用于Web开发，并可以嵌入到HTML中。

```
PHP |
1  <?php
2  echo "Hello, World!";
3  ?>
```

ASP.NET — a server-side web application framework developed by [Microsoft](#).

一个由微软开发的**服务器端**Web应用程序框架，支持多种编程语言，并提供强大的开发工具和库。

Bootstrap — a **free** and **open-source** **CSS** framework directed at responsive, mobile-first front-end web development with [HTML](#), [CSS](#) and [Javascript](#)

一个免费的开源CSS框架，针对响应式和移动优先的前端Web开发，使用HTML、CSS和JavaScript。Bootstrap提供了丰富的UI组件和布局工具，简化了前端开发过程

响应式设计 (Responsive Design)

响应式设计是一种Web设计方法，旨在创建能够自动适应不同设备屏幕尺寸和分辨率的网站。这种设计方法确保网站在桌面电脑、平板电脑、智能手机等设备上都能提供良好的用户体验。

移动优先设计 (Mobile-First Design)

移动优先设计是一种设计策略，首先考虑移动设备的用户体验，然后再扩展到桌面设备。这种方法反映了当前移动互联网使用量逐渐超过桌面互联网的趋势。

Django	Ruby on Rails	Laravel	ASP.NET
Python	Ruby	PHP	C# Visual Basic .NET
FULL-STACK、Server-side、Open-source、free、MVC...			

Common web framework functionality

- URL routing
- HTML, XML, JSON, and other output format templating
- Database manipulation
- Security against Cross-site request forgery (CSRF) and other attacks
- Session storage and retrieval

- URL routing

URL路由– 将用户请求的URL映射到应用程序中的特定处理程序的过程

- HTML, XML, JSON, and other output format templating

模板引擎通过插入动态数据来生成所需的格式内容

- Database manipulation

数据库操作 涉及与数据库的交互，包括数据的创建、读取、更新和删除（CRUD操作）。应用程序框架通常提供ORM（对象关系映射）工具，以简化数据库操作，使开发人员无需编写复杂的SQL查询。

- Security against Cross-site request forgery (CSRF) and other attacks

安全性是Web应用程序开发中的关键问题。框架通常内置了一些安全措施，以防止常见的攻击，如CSRF（跨站请求伪造）和XSS（跨站脚本攻击）。这些安全措施包括验证请求的来源、防止恶意脚本注入等。

- Session storage and retrieval

会话存储和检索

会话管理 涉及存储和检索用户会话数据。会话数据可以包括用户的登录状态、购物车内容等。框架通常提供会话管理工具，以便开发人员能够方便地管理用户会话，确保数据的一致性和安全性。

Not all web frameworks include code for all of the above functionality.

- For example, the Django web application framework includes an Object Relational Mapping (ORM) layer that abstracts relational database read, write, query, and delete operations.

However, Django's ORM cannot work without significant modification on non-relational databases such as MongoDB.

- Some other web frameworks such as Flask and Pyramid are easier to use with non-relational databases by incorporating external Python libraries–整合外部python库。

二、Database Management Systems

- Your web application needs a **database** to store its data, either relational or non-relational databases.
- Examples of DBMS:
 - MongoDB - an open-sourced NoSQL database.
 - PostgreSQL - a popular open-sourced SQL database.
 - MySQL - another popular open-sourced SQL database.
 - Oracle - an enterprise SQL database.
 - SQL Server - an SQL server manager created by Microsoft.

Database : Store its data --- relational or non-relational databases.

▼ 关系型数据库 (Relational Databases) 和非关系型数据库 (Non-relational Databases) 是两种不同类型的数据库管理系统

特性	关系型数据库 (Relational Databases)	非关系型数据库 (Non-relational Databases)
定义	使用表格 (表) 来组织和存储数据	使用多种不同的数据模型, 如键值对、文档、列族、图等
数据结构	结构化数据, 具有固定模式 (Schema)	非结构化或半结构化数据, 模式灵活
关系	通过键建立表之间的关系	不强调表间关系, 数据模型多样
查询语言	使用SQL (结构化查询语言)	使用不同的查询语言和API
事务支持	支持事务, 确保数据一致性和完整性	一些数据库支持事务, 但不如关系型数据库完善
扩展性	垂直扩展 (增加硬件资源)	水平扩展 (增加更多节点)
性能	适合处理复杂查询	适合处理大规模数据和高并发访问
常见数据库	MySQL、PostgreSQL、SQLite、Oracle等	MongoDB、Cassandra、Redis、Neo4j等

关系型数据库适用场景:

- 数据结构较为稳定, 具有明确模式。
- 需要复杂查询和事务支持。
- 数据之间存在复杂的关系, 需要通过外键关联。

非关系型数据库适用场景:

- 数据量大, 数据结构灵活多变。
- 需要高并发读写操作和快速数据访问。
- 数据类型多样, 如文档、图、键值对等。

Examples(DBMS):

- **MongoDB** – an open-sourced **NoSQL** database. – NoSQL (非关系型数据库)
- **PostgreSQL** – a popular open-sourced SQL database.
- **MySQL** – another popular open-sourced SQL database.
- **Oracle** – an **enterprise** SQL database.
- **SQL Server** – an SQL server manager created by **Microsoft**.

数据库管理系统 (DBMS) 是一种**软件**, 主要负责数据的存储、组织和管理, 以及提供数据访问接口 (如SQL查询) 。

服务器是一种计算机硬件或软件系统, 提供网络服务和资源。它可以处理客户端的请求, 执行业务逻辑, 并返回响应

开源: MongoDB, PostgreSQL, MySQL

不开源: Oracle, SQL Server.

三、Client & Server

1. Client (side)

- This party **requests** pages from the **Server**, and displays them to the user. In most cases, the client is a **web browser**.
 - The **User** - The user *uses* the **Client** in order to surf the web, fill in forms, watch videos online, etc.
 - It's you and me when we visit <http://google.com>.
- Client can be desktop computers, tablets, or mobile devices.
- There are typically multiple clients interacting with the same application stored on a server.

This party **requests** pages from the **Server**, and displays them to the user.

In most cases, the client is a **web browser**.

(1) 请求数据

(2) 展示给用户 User使用Client浏览网页

2. Server (side)

- This party is responsible for **serving** pages.
- A server is where the application code is typically stored.
- Requests are made to the server from clients, and the server will gather the appropriate information and **respond to those requests**.

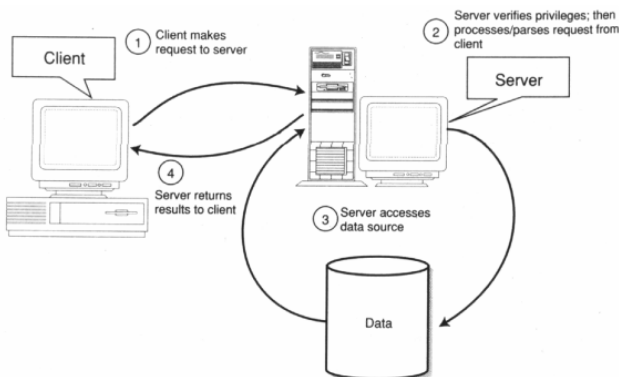
This party is responsible for **serving** pages.

“party” 指的是负责提供网页服务的一方

“Serving pages” 是指**Web服务器将网页内容发送给用户的过程**。当用户在浏览器中请求一个网页时，Web服务器会响应这个请求，将相应的HTML文件、CSS文件、JavaScript文件和其他资源发送给用户的浏览器，以便浏览器能够渲染并显示这个网页。

- A server is where the application code is typically stored.
- Server **respond to those requests**

3. Client–Server Model



- a. User 打开浏览器并浏览网页；
- b. Client向Server发出Request；
- c. Server响应Request，并发送信息至Client；

(Replies the client with some meta-data (called *headers*), followed by the page's source.– 服务器首先会回复一些元数据（称为头部信息），然后再发送网页的源代码）

1. **Meta-data (headers)**: 头部信息包含了一些关于请求和响应的元数据, 这些信息不会显示在网页中, 但对浏览器处理请求和响应非常重要。例如, 头部信息可以包含:
 - **Content-Type**: 指定响应内容的类型 (如text/html表示HTML文档)。
 - **Content-Length**: 指定响应内容的长度。
 - **Server**: 指示服务器软件的信息。
 - **Date**: 响应生成的时间。
 - **Cache-Control**: 缓存控制信息。
2. **Page's source**: 网页的源代码, 即HTML文件以及可能的CSS和JavaScript文件。这些文件构成了用户在浏览器中看到的网页内容。

d. Client收到page's source, **renders** it into a human viewable website.

渲染: 指的是浏览器将收到的网页源代码转换成可以在人眼中看到和交互的网页。

渲染过程

1. **解析HTML**: 浏览器首先解析HTML文档的结构, 构建DOM (文档对象模型) 树。
2. **解析CSS**: 同时, 浏览器解析CSS文件和内嵌的样式规则, 构建CSSOM (CSS对象模型) 树。
3. **解析JavaScript**: 浏览器会执行JavaScript代码, 可能会进一步修改DOM和CSSOM。
4. **布局 (Layout)**: 浏览器结合DOM和CSSOM, 计算每个元素在页面中的位置和尺寸。
5. **绘制 (Paint)**: 浏览器将各个元素绘制到屏幕上, 生成最终的视觉输出。

“Render”指的是浏览器将收到的HTML、CSS和JavaScript代码解析、计算和绘制成可以在屏幕上看到的网页内容的过程。

四、Front-end and back-end

- The **front-end** is comprised of HTML, CSS, and JavaScript. This is how and where the website is shown to users.
- The **back-end** is comprised of your **server and database**. It's the place where functions, methods, and data manipulation happens that you don't want the clients to see

对比 Client and Server

- Client: This party **requests** pages from the Server, and displays them to the user.
- Server: This party is responsible for **serving** pages.

前端 (front-end): 特指用户界面的开发和实现, 主要关注用户体验和视觉设计。

客户端 (client): 用户设备上的应用程序, 通过网络与服务器交互, 获取和呈现数据。

五、Client-side programming

- Client-side programming is writing **code** that will **run on the client**, and is **done in languages**

that can be executed by the browser, such as **JavaScript**. –making web pages interactive.

- Client-side (i.e. frontend) web development involves everything users see on their screens.

Some major frontend technology stack components:

- Hypertext Markup Language (**HTML**) and Cascading Style Sheets (**CSS**).

HTML tells a browser how to display the content of web pages,

CSS styles that content.

- **Bootstrap** is a helpful framework for managing HTML and CSS.
- There are many JavaScript libraries (such as jQuery, Vue.js, React.js – maintained by Meta) for faster and easier web development.
- **Client side** programming has mostly to do with the **user interface(UI)**, with which the user interacts. 客户端编程 (Client-side programming) 主要涉及用户界面 (UI) , 即用户与之交互的部分
- **Its main tasks are:**
 - *Validating input* (Validation **must be** done in the **server**. A redundant validation in the client could be used to avoid server calls避免对服务器的调用 when speed is very critical.) 客户端进行冗余验证可以提高速度, 因为它可以在用户提交数据之前捕获错误, 减少与服务器的交互次数。这在速度非常关键的情况下特别有用。
 - *Animation* 动画-动态效果
 - *Manipulating UI elements*
 - *Applying styles*
 - *Some calculations are done when you don't want the page to refresh so often* 在客户端进行一些计算可以避免页面刷新, 提高用户体验。例如, 表单中的数据验证、购物车的总价计算等, 可以在客户端完成, 而不需要每次都与服务器交互。

六、Server-side programming

- The server side isn't visible to users, but it powers the client side.
- Server-side programming is writing code that **runs on the server**, using languages supported by the server.
- Server-side programming is used to create the logic of websites and applications.
- Frameworks for programming languages offer lots of tools for simpler and faster coding.

Some of the popular programming languages and their major frameworks :

- Ruby (Ruby on Rails)
- Python (Django, Flask)
- PHP (Laravel)
- C# (ASP.NET)

- Typical uses of server-side programming

- Process user input.
- Display pages.
- Structure web applications.

构建（或结构化）Web应用程序。这涉及到如何设计和组织Web应用程序的各个组件，使其高效、可维护和易于扩展。 服务器处理客户端请求并与数据库交互。

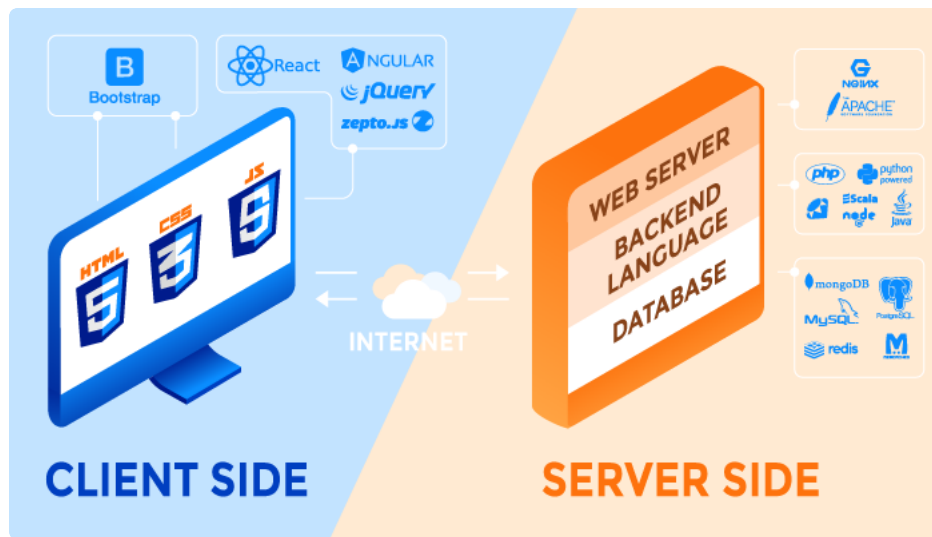
- Interact with permanent storage (SQL, files).

Querying the database

Insert and update information onto the database

▼ Display Page在服务器端编程与客户端编程的差别

特性	服务器端页面展示 (Server-side rendering)	客户端编程 (Client-side programming)
处理位置	页面内容在服务器上生成，然后发送到客户端	页面内容在客户端（浏览器）通过JavaScript动态生成
处理语言	PHP、ASP.NET、Ruby on Rails、Node.js等	HTML、CSS、JavaScript及其框架（如React、Angular等）
SEO 友好性	较高，搜索引擎更容易抓取和索引内容	较低，搜索引擎对动态生成的内容抓取有难度
初始加载速度	快，内容已生成，用户可以快速看到完整页面	慢，初次加载需下载大量JavaScript文件
服务器负担	高，所有请求需要服务器处理	低，部分处理在客户端进行，减少服务器工作量
交互性	交互性较差，频繁与服务器通信	交互性强，用户操作可以立即反馈
数据安全性	高，数据在服务器端处理，减少前端数据泄露风险	低，部分逻辑在客户端处理，存在安全隐患
适用场景	需要快速初始加载、SEO优化且安全性要求高的场景	需要高交互性、动态更新且希望减轻服务器负担的场景



附

计算机编程开发框架

计算机编程开发框架是一个为软件开发过程提供基础结构和支撑的环境集合。

框架通过这一基础设施的支持，使开发者可以专注于开发软件的独特功能部分，而不用从头编写所有底层的代码。—— 代码复用，降低软件开发的复杂性和时间成本。

例，模型-视图-控制器 (MVC) 架构，将数据模型、用户界面、用户输入，分离成独立的模块。

一、框架的通用结构

核心库，提供基础功能支持；

API接口，使得与框架进行交互成为可能；

插件系统，允许扩展框架核心功能；

文档，为开发者提供使用指南和参考资料

二、不同类型的框架

eg. Web框架、移动应用框架、游戏开发框架...

三、框架与库-控制流程

通常，框架具有更强的控制力，它规定了程序的骨架和执行流程；

而库提供了一组功能，开发者可以自由选择并控制这些功能来完成特定任务。

框架提供了一种标准化的方法来构建和部署应用，库更多的是作为一个资源的集合

1. 前端开发框架：例如React、Angular和Vue.js等框架，它们专注于构建交互式的用户界面和单页面应用程序（SPA）。
2. 后端开发框架：例如Node.js、Django和Ruby on Rails等框架，用于构建服务器端应用程序和API。
3. 移动应用开发框架：例如React Native和Flutter等框架，用于跨平台开发移动应用程序。
4. 数据科学和机器学习框架：例如TensorFlow、PyTorch和Scikit-learn等框架，用于构建和训练机器学习模型。
5. 数据库和ORM框架：例如MySQL、PostgreSQL、MongoDB和Hibernate等框架，用于简化数据库操作和对象关系映射。